

1. University Course Registration System

- Design and implement a client–server application for managing student enrollment.

Two user types: Student, Academic Administrator. The system stores all data in a database.

Student actions:

- View available courses
- Register for a course
- Drop a course
- View personal timetable
- Export timetable as JSON

Administrator actions:

- CRUD on courses
- CRUD on professors
- Approve overload requests
- View enrollment statistics
- Import course lists from XML

2. Medical Appointment Scheduling System

- Design a client–server application for a medical appointment scheduling system

Two user types: Patient, Doctor. The system stores all data in a database.

Patient actions:

- View available appointment slots
- Book an appointment
- Cancel an appointment
- View medical history
- Update personal data

Doctor actions:

- View daily schedule
- Add medical notes
- Mark appointment as completed
- Manage available time slots
- Export patient history as XML

3. Hotel Booking Management System

- Design a client–server application for a hotel booking management system

Two user types: Guest, Hotel Manager. The system stores all data in a database.

Guest actions:

- Search available rooms
- Make a reservation
- Cancel a reservation
- View reservation history
- Receive booking confirmation in JSON

Manager actions:

- CRUD on rooms
- CRUD on seasonal prices
- Approve special requests
- View occupancy statistics
- Import room lists from XML

4. Online Forum (Mini-StackOverflow)

- Design a client–server application for an Online Forum (Mini-StackOverflow).

Two user types: Regular User, Moderator. The system stores all data in a database.

Regular User actions:

- Create a question
- Post an answer
- Vote on posts
- Edit own posts
- Search questions

Moderator actions:

- Delete inappropriate posts
- Suspend users
- Edit tags/categories
- Approve flagged posts
- Export activity logs as JSON

5. Real-Time Car Tracking System

- Design and implement a client–server application for monitoring a fleet of cars.

The system does not require any map visualization. The server stores and processes all vehicle data, while the client allows drivers and managers to interact with the system.

Driver actions:

- Update the car's current location as plain text (e.g., street name, checkpoint code, or predefined zone)
- Mark the vehicle as *active* or *inactive*
- Report a technical issue (engine warning, flat tire, etc.)
- View assigned routes
- Confirm completion of a route segment

Fleet Manager actions:

- View the list of all vehicles with their latest reported locations
- Assign routes to drivers
- Generate a daily activity summary (TXT or XML)

The system stores all data in a database and supports simple text-based location tracking rather than map rendering.

6. Chat Application with Bot Integration

Design and implement a client–server chat application where users communicate only with a chat bot. The client sends messages; the server processes them and returns responses, including bot replies. The system stores all data in a database.

Client capabilities:

- Send text messages to the server
- Send predefined bot commands (e.g., /help, /time, /joke)
- View chat history stored on the server
- Update personal profile information

Server capabilities:

- Receive and store all messages in the database
- Process bot commands and generate automated responses
- Export chat logs in XML or JSON format

The bot logic is simple (rule-based or keyword-based), and no real AI integration is required.

7. Continuous Integration Mini-Platform (Jenkins-Lite)

Design and implement a simplified CI/CD platform.

The system runs local build jobs (no Git integration) and displays their status.

Two actors interact with the system: Developer and Build Administrator.

Developer actions:

- Can add new projects and set build configuration (e.g. build or test command)
- Trigger a build for a selected project
- View build history and status (success/failure), see a simple graph (text-based or basic chart, not real-time dashboards)
- Download build artifacts generated by the server

Build Administrator actions:

- Import/export job definitions in JSON (project metadata, build configuration)

The server executes builds locally (e.g., running a script or command), stores results in the database, and updates the dashboard with green/red indicators.

8. Insurance Claim Processing System

- Design and implement a client-server application for managing insurance claims.

Two user types: Client and Insurance Agent.

The system exchanges structured data with external partners (e.g., hospitals, repair shops) using XML-based messages.

Client actions:

- Submit a new insurance claim (accident, medical, property)
- Track the status of submitted claims
- Receive claim summaries in XML format

Insurance Agent actions:

- Review and approve/reject claims
- Request additional documents from clients
- Update claim status and settlement amounts
- Generate monthly claim statistics (TXT or XML)

9. Emergency Alert Broadcasting System for University Campus

- Design and implement a client-server application used by a university campus to distribute emergency alerts to different faculties, dormitories, and administrative buildings.

The server acts as the central point where alerts are created, routed, and broadcast to all connected clients or to specific campus groups.

Two user types: Campus Security Operator and Campus Receiver (installed in each building or department).

Campus Security Operator actions:

- Create a new emergency alert (fire, evacuation, severe weather, technical outage, security threat), save alerts to DB
- Choose the broadcast scope: entire campus or specific buildings/departments

- Update or cancel an active alert

Campus Receiver actions:

- Receive broadcast alerts in real time from the server
- Acknowledge receipt of an alert
- Save alert logs to DB

10. Bug Reporting and Tracking System

- Design and implement a client-server application used inside a software development team to report, track, and manage software bugs.

The server stores all bug reports, assigns them to developers, and maintains their status throughout the lifecycle.

Two user types: Developer and Administrator.

Developer actions:

- Submit a new bug report, including title, description, severity, photo and steps to reproduce
- View the list of bugs
- Update the status of a bug (e.g., new, in progress, fixed, cannot reproduce)

Administrator actions:

- View all reported bugs across the system
- Import bug reports from an external XML file (e.g., reports from another team or tool)